DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40J

USAREUR-AF

CONCEPTS GUIDE — TM-40J COMPANION

DATA PROGRAM MANAGER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

CONCEPTS GUIDE — TM-40J COMPANION

DATA PROGRAM MANAGER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide develops the mental models required to manage data pipelines, programs, and portfolio health on MSS effectively. It is a prerequisite companion to TM-40J and is intended to be read before beginning TM-40J task instruction.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. The Data PM's Role on MSS
2. Thinking in Pipelines and Dependencies
3. Health vs. Status — The PM's Monitoring Framework
4. Milestone and Milestone Risk Thinking
5. Managing Technical Debt in a Data Portfolio
6. Stakeholder Management for Data PMs
7. Governance as a PM Tool, Not a PM Obstacle
8. Portfolio Visibility on MSS
9. Common Data PM Failure Modes

DRAFT

PREFACE

BLUF: This guide is for data program managers who will be accountable for a portfolio of data products on the Maven Smart System (MSS). It does not teach Foundry mechanics. It develops the judgment to manage them.

Most failures on MSS data programs are not technical. They are organizational and conceptual: a PM who accepted a delivery timeline before confirming data access, a project that shipped without governance sign-off and immediately generated downstream data quality complaints, a dashboard that has been in production for eight months but no one is certain who owns it. These failures share a common root: the PM did not have a coherent mental model of what they were managing.

This guide builds that mental model across nine areas of PM practice. Read it linearly before beginning TM-40J task instruction. Return to individual sections as needed when specific challenges arise in practice.

SECTION 1: THE DATA PM'S ROLE ON MSS

1-1. What a Data PM Owns

A data program manager on MSS does not own code. They do not own pipelines, Object Types, or Foundry Workshop applications in the technical sense. What they own is accountability for outcomes: a portfolio of data products that are working, governed, and meeting operational requirements.

That distinction matters. A builder owns a pipeline. A data steward owns naming conventions and governance. A software engineer owns code quality. The PM owns all of them in aggregate — not by doing the work, but by ensuring the work adds up to something operationally useful and sustainable.

In practice, the PM's primary outputs are:

DRAFT

Output	Description
Delivery milestones	Scoped, resourced, and tracked to completion
Portfolio health visibility	Aggregate view of what is working, what is degraded, what is at risk
Stakeholder alignment	Operational users and technical producers working toward the same definition of done
Governance compliance	Data products built and promoted through proper C2DAO channels
Debt awareness	Identification and prioritization of technical and governance debt accumulating in the portfolio

The PM manages across people, pipelines, and products. Not just projects. The distinction is important: a project has a start and end date. A data product has a lifecycle — it requires ongoing ownership, maintenance decisions, and eventual retirement. A PM who thinks only in projects will consistently under-resource the sustain phase.

1-2. What the PM Does Not Own

The PM does not own technical architecture decisions. When a builder proposes a particular pipeline design, the PM's role is not to approve or reject the design on technical merit — that judgment belongs to the engineer or data steward. The PM's role is to ensure the design was reviewed by the right people, that it meets governance requirements, and that the timeline is realistic given the approach chosen.

The PM does not own data quality. Data quality is a production standard owned by data stewards and enforced through governance. The PM creates the conditions for quality (adequate review time, proper sign-off, thorough testing windows) but cannot substitute their judgment for domain expertise on whether a particular dataset is correct.

The PM does not own access decisions. MSS access control decisions — who can read, write, or promote data products — belong to the USAREUR-AF C2DAO. The PM coordinates access requests and ensures they are processed in time to support project schedules; they do not grant access unilaterally.

1-3. The PM and C2DAO Governance

The C2DAO (Command and Control Data Architecture Office) is the USAREUR-AF authority for MSS data governance. It sets naming conventions, owns the promotion gate criteria, maintains the master Ontology design, and adjudicates data stewardship disputes.

The PM does not report to C2DAO, but every data product the PM delivers must pass through C2DAO governance. This creates a structural dependency: every project schedule must account for C2DAO review cycles. A PM who treats C2DAO review as optional until late in a project is building a schedule that

will slip.

The productive relationship between the PM and C2DAO is collaborative, not adversarial. The C2DAO wants data products to succeed and reach production. Their role is to ensure those products do not create downstream problems — naming collisions, unsanctioned Object Types, pipelines with no documented owner. The PM who engages C2DAO early, keeps them informed of upcoming builds, and incorporates governance checkpoints into project plans will move faster, not slower.

SECTION 2: THINKING IN PIPELINES AND DEPENDENCIES

2-1. The Dependency Chain

No data product on MSS is a standalone artifact. Every product the PM is responsible for sits at the end of a dependency chain. Understanding that chain — and being able to trace it — is a core PM survival skill.

The standard dependency chain on MSS runs:

```
Source System → Ingestion Pipeline → Raw Dataset → Transformed Dataset
→ Ontology Object Type → Workshop Application → Operational User
```

A readiness reporting dashboard visible to a V Corps staff officer has: - A source system (GCSS-Army or a unit-submitted data feed) - An ingestion pipeline (a Foundry transform that pulls or receives that feed) - A raw dataset (the ingested, unmodified data in Foundry) - A transformed dataset (cleaned, normalized, structured for analytical use) - An Ontology Object Type (the abstraction that makes the data queryable across tools) - A Workshop application (the dashboard the staff officer sees)

If the source system stops sending data, the dashboard goes stale. If the transform breaks, the dashboard may display corrupt data. If the Ontology Object Type gets modified without versioning, every application that depends on it may fail silently.

The PM who does not understand this chain cannot triage. When a user reports "the dashboard is wrong," a PM without dependency awareness escalates to the builder and waits. A PM with dependency awareness asks: is the source feed arriving? Is the ingestion pipeline running? Did any transforms fail in the last 24 hours? Has anyone modified the Ontology Object Type recently? That triage reduces mean time to resolution.

2-2. Dependency Mapping as a PM Practice

Every significant data product in the portfolio should have a dependency map. This does not need to be elaborate. A simple three-column table is sufficient for most products:

Layer	Asset Name	Owner	Last Validated
Source	GCSS-A feed (LOGSA pull)	21st TSC G4 data steward	2026-02-15
Ingestion	`gcss_a_ingest_transform`	Builder, SSG Petrov	2026-02-01
Raw Dataset	`gcss_a_raw`	Data Steward, CW2 Flores	2026-02-15
Transformed	`equipment_readiness_v3`	Builder, SSG Petrov	2026-01-30
Object Type	EquipmentReadinessRecord	C2DAO Ontology team	2026-01-15
Application	LOGSTAT Dashboard v2	PM, MAJ Reyes	2026-02-10

This table answers three questions the PM will be asked regularly: Who owns each piece? When was it last confirmed working? What is the chain of custody for this data product?

Maintain this table for every tier-1 product in the portfolio. Update it after every significant change. A dependency map that is six months stale is almost as dangerous as no map at all.

2-3. When Dependencies Break

Dependencies break in three ways: silently, noisily, or slowly.

A noisy break is easy: a pipeline fails with an error, Foundry logs the failure, the builder gets alerted. The PM's role is to ensure resolution time is within SLA and to communicate status to affected users.

A silent break is dangerous: a pipeline runs without error but produces incorrect output. The source system changed its schema and the transform no longer maps correctly. The data looks plausible but is wrong. Silent breaks often go undetected until an operational user acts on bad data. The defense against silent breaks is data quality checks embedded in transforms and a culture of user reporting.

A slow break is the most common: data that was timely yesterday is now 18 hours stale. A pipeline that ran in four minutes now takes 40. A dashboard that loaded instantly now takes 90 seconds. These are not failures — they are degradations. The PM who monitors only for failures will miss slow breaks entirely until they become crises.

SECTION 3: HEALTH VS. STATUS — THE PM'S MONITORING FRAMEWORK

3-1. The Distinction

Status answers: is it running? Health answers: is it doing what it is supposed to do?

These are different questions, and conflating them is a common PM error. A pipeline can have a green status in Foundry's monitoring interface while producing stale, low-quality data that no one is using. That pipeline has a health problem, not a status problem. Reporting its status as green misrepresents the actual state of the portfolio.

Every PM who has briefed a stakeholder on portfolio health using only status indicators has, at some point, been surprised when a "green" product generates user complaints. The framework below prevents that.

3-2. Three Health Dimensions

Assess portfolio health across three dimensions:

Availability (Is it running?) - Are pipelines executing on schedule? - Are datasets refreshing within SLA? - Are Workshop applications accessible and loading within acceptable response time? - Are ingestion feeds arriving from source systems?

Quality (Is the data correct?) - Are data quality checks passing? - Is data completeness within accepted thresholds? - Are there anomalies or outliers that suggest upstream corruption? - Has the data been validated against a known-good source recently? - Do users report that the data matches their operational reality?

Utility (Is it being used for its intended purpose?) - Are users accessing the application at the expected frequency? - Are the operational questions the product was designed to answer actually being answered with it? - Has the user base shrunk since deployment? Why? - Has the operational requirement the product was built to support changed since it was built?

3-3. Applying the Framework: A V Corps Example

Consider a readiness reporting pipeline built to support V Corps daily battle rhythm. The pipeline has been running for seven months.

DRAFT

Dimension	Question	Finding
Availability	Is the pipeline running?	Yes, daily at 0300Z
Availability	Are datasets refreshing on schedule?	Yes, within 15 minutes of execution
Quality	Are data quality checks passing?	Partially — completeness check fails for 3 subordinate units
Quality	Do users trust the data?	Unknown — no user feedback collected since month 2
Utility	Is it being used in battle rhythm?	Unclear — application access dropped 60% in month 5
Utility	Is it still meeting operational requirements?	Unknown — S3 changed readiness reporting format in month 4, product not updated

Status: green. Health: degraded. The PM who only checks status misses four significant findings. The PM who checks health across all three dimensions has an actionable picture.

3-4. Health Review Cadence

Conduct a full three-dimension health review for all tier-1 products monthly. For products in active development or recent production release, conduct availability and quality checks weekly. Utility checks should include direct user engagement at least quarterly — automated usage metrics alone do not capture whether the product is meeting its operational purpose.

SECTION 4: MILESTONE AND MILESTONE RISK THINKING

4-1. How Data Projects Actually Fail

Data projects fail in predictable ways. The PM who has seen one MSS project fail has seen the pattern of how most will fail. The five most common failure modes, in order of frequency in USAREUR-AF data programs:

1. **Data access lead time was underestimated.** The project assumed source system access would be granted within two weeks. It took eight. The schedule was never adjusted.
2. **Ontology design changed late.** The Object Type the project was building toward was modified by another team mid-build. The builder had to rework transforms. Three weeks were lost.
3. **Governance approval took longer than planned.** C2DAO review was scheduled for the last two weeks of the project. Review identified naming violations that required rework. Delivery slipped one month.

4. **User acceptance failed at delivery.** The product was built against requirements captured in month one. By delivery in month four, the operational requirement had changed. Users rejected the product at demonstration.
5. **Key personnel departed.** The sole builder on the project PCS'd in month three. Knowledge transfer was incomplete. The replacement builder spent six weeks reconstructing context.

These are not unpredictable risks. They are expected conditions for data programs in a theater environment. Build them into every project plan.

4-2. Milestones vs. Checkpoints

A milestone marks delivery: the pipeline is built, the dataset is available, the dashboard is deployed. A checkpoint verifies that a delivery is working as intended.

Most project plans have milestones. Fewer have checkpoints. This is a mistake.

The difference matters in practice:

Milestone	Paired Checkpoint
Data ingestion pipeline deployed	Data completeness check passes; latency within SLA; three business days of clean execution confirmed
Ontology Object Type created	Object Type reviewed and approved by C2DAO data steward; no naming violations; link types validated
Workshop application delivered	User acceptance testing complete with actual operational users; access permissions confirmed; usage baseline established
Project close-out	Product owner identified; maintenance responsibility documented; dependency map filed in project record

A milestone without a checkpoint is a declaration. A milestone with a checkpoint is a confirmation. Operational programs require confirmation.

4-3. Building Risk into Schedules

Every MSS data project schedule should explicitly account for:

- **Data access lead time:** Add three to six weeks for source system access requests, particularly for feeds originating outside USAREUR-AF (LOGSA, DCSA, external theater partners). Do not begin pipeline builds against data that has not yet been confirmed accessible.
- **C2DAO review cycles:** Add two to three weeks for governance review at each promotion gate (development → staging → production). Engage C2DAO before development begins to preview the product design and surface objections early.

- **Ontology coordination buffer:** If the project requires new or modified Object Types, add one to two weeks for Ontology coordination with other teams who may have dependencies on the same types.
- **User acceptance buffer:** Add one to two weeks after delivery for user acceptance testing with actual operational personnel. Do not accept PMO sign-off as user acceptance. They serve different purposes.
- **Personnel continuity risk:** For projects longer than three months, document the bus factor — the number of team members whose departure would critically degrade the project. Maintain knowledge transfer artifacts for any single-point-of-failure personnel.

SECTION 5: MANAGING TECHNICAL DEBT IN A DATA PORTFOLIO

5-1. What Technical Debt Looks Like in Data Programs

Technical debt in a data portfolio is not always visible. It accumulates in places that do not generate immediate alerts:

- Pipelines built quickly during an exercise (DEFENDER, Spring Storm) that were never refactored or documented, now running in production as semi-permanent products.
- Object Types created by a builder who has since PCS'd, with unclear ownership, no documentation, and several downstream applications that depend on them but may not know it.
- Dashboards deployed for a specific operation six months ago, now receiving near-zero traffic but still scheduled to refresh daily, consuming compute resources.
- Transforms that were written to accommodate a quirk in a source system that has since been fixed, but the workaround was never removed and now produces incorrect results.
- Governance documentation that exists but is six months out of date, creating a false impression of compliance.

Each of these is a form of debt. Unlike code debt, data portfolio debt can directly affect operational users: the stale dashboard still referenced in a unit SOP, the Object Type whose schema no longer matches reality.

5-2. Identifying and Quantifying Debt

Conduct a portfolio debt audit twice per year. For each data product in the portfolio, answer five questions:

Question	Debt Indicator
Does it have a named, reachable owner?	No owner = governance debt
Is its documentation current (within 90 days of last change)?	Stale docs = documentation debt
Is it being actively used?	<10% of expected usage = potential retirement candidate
Were its governance artifacts (naming, promotion sign-off) completed?	Missing artifacts = governance debt
Does it have any known quality issues that have not been remediated?	Open quality issues = quality debt

Assign each product a debt level: Low (0-1 flags), Medium (2-3 flags), High (4-5 flags). High-debt products require remediation plans or retirement decisions.

5-3. Debt vs. New Capability: Making the Case

Leadership will consistently pressure the PM to deliver new capability. Debt remediation does not appear in a commander's dashboard. It does not generate a milestone brief. It is invisible — until it causes a production failure at 0200Z during a major exercise.

The PM's job is to make the debt visible and make the cost of not addressing it concrete.

Frame the debt case to leadership using operational risk language, not technical language:

- "The readiness reporting pipeline for 21st TSC has no documented owner, no rollback procedure, and a known data quality issue affecting completeness for three subordinate units. If it fails during DEFENDER, we cannot reconstitute it without four to six days of rebuild time. The risk is an operational reporting gap during peak exercise tempo."

That argument is more persuasive than: "We have technical debt in our data portfolio that needs to be addressed."

Propose a debt-to-new-capability ratio for sprint allocation: 20% of builder capacity dedicated to debt remediation is a defensible baseline. Adjust based on audit findings. When the portfolio audit surfaces high-debt products, negotiate a temporary increase.

DRAFT

SECTION 6: STAKEHOLDER MANAGEMENT FOR DATA PMS

6-1. Two Stakeholder Classes

The data PM serves two fundamentally different stakeholder classes simultaneously.

Operational users — Commanders, staff officers, NCOs who consume data products to make decisions. Their concern is whether the product works, whether it answers their operational question, and whether they can trust it. They do not understand pipeline architecture. They do not need to. They need accurate, timely, accessible data.

Technical producers — Builders, engineers, data stewards, C2DAO personnel who design and build the products. Their concern is whether the requirements are stable, whether the architecture is sound, whether there is adequate time for quality governance. They understand pipeline architecture. They do not always understand why a commander needs something by Tuesday.

The PM's translation function runs in both directions:

- From operational users to technical producers: "The S3 needs a readiness view that shows equipment status by subordinate unit within V Corps. The access population is field-grade officers and above. The refresh requirement is daily by 0600Z."
- From technical producers to operational users: "The dashboard you need requires data from three source systems, two of which are outside USAREUR-AF. Initial delivery will take six weeks. You will see a prototype by week three."

The PM who fails to translate in both directions creates a gap that manifests as missed requirements, schedule surprises, and post-delivery rejection.

6-2. The Delivery Tension

The central tension in data PM stakeholder management: operational users want delivery fast; technical producers need time to build quality products that will not fail in production.

Both positions are legitimate. The PM does not resolve this tension by choosing a side. The PM manages it through:

Staged delivery. A working prototype with partial data, delivered on the user's timeline, allows early feedback and reduces the risk of late-stage requirement changes. Frame it explicitly as a prototype. Do not let a prototype drift into production without completing governance.

Transparent constraints. Users who understand why a delivery takes time are more patient than users who are simply told to wait. "Data access from LOGSA requires a formal request through NETCOM that takes three to four weeks" is a concrete constraint. Users can accommodate it. "It takes time" is not a constraint — it is an excuse.

Scope negotiation. When a full-scope delivery cannot meet the user's timeline, negotiate scope reduction. What is the minimum viable product that meets 80% of the operational requirement? Deliver that on time, then scope the remainder into a follow-on release.

6-3. Managing a 21st TSC Ontology Coordination Scenario

Consider this scenario: V Corps G4 requests a readiness dashboard. The product requires a new Object Type that will also be used by 21st TSC for their own logistics pipeline. The 21st TSC data steward has existing views on how that Object Type should be structured.

Without stakeholder management, this becomes a conflict that stalls both projects. With stakeholder management, the PM:

1. Identifies the shared dependency early — before either team begins building.
2. Convenes a coordination session with V Corps G4, 21st TSC data steward, and C2DAO Ontology team.
3. Facilitates agreement on the Object Type schema that meets both teams' needs, or negotiates a clear owner and consumer relationship.
4. Documents the outcome and incorporates the coordination timeline into both project schedules.
5. Follows up with C2DAO to ensure the agreed schema receives formal governance approval before any build begins.

The PM does not design the Object Type. The PM creates the conditions for the right people to make the right decision on time.

SECTION 7: GOVERNANCE AS A PM TOOL, NOT A PM OBSTACLE

7-1. The Governance Temptation

Under schedule pressure, governance looks like friction. The C2DAO review takes three weeks. The naming convention review catches issues that require rework. The data steward sign-off requires a meeting that takes two days to schedule. The PM who is already late on a milestone will be tempted to move to production and close the governance loop afterward.

This is a predictable mistake with predictable consequences.

A data product promoted to production without governance sign-off: - May have naming violations that create collision problems when another team builds against the same Ontology. - May lack a documented owner, creating an orphaned asset that no one will maintain. - May bypass quality checks that would have

caught data correctness issues before operational users encountered them. - Will require retroactive remediation that costs more time than the review would have taken — and must happen while the product is live, affecting operational users.

The PM who bypasses governance to deliver faster is borrowing time, not saving it.

7-2. Using Governance Checkpoints as Milestone Gates

The productive reframe: governance checkpoints are natural milestone gates. They represent the moments in a project when the right authorities confirm that the work so far is sound before the next phase begins.

Map governance events to project milestones:

Governance Event	Natural Project Gate
C2DAO Ontology design review	Gate before beginning Ontology build
Naming convention validation	Gate before promoting to staging
Data steward sign-off on quality checks	Gate before user acceptance testing
C2DAO promotion approval	Gate before production release
Data product ownership documentation	Gate before project close-out

This mapping has two benefits. First, it prevents governance events from being discovered late — when they are on the schedule, they happen when planned. Second, it gives the PM natural hold points at which to validate the project before committing to the next phase. A governance review that surfaces a naming issue before the staging build is a feature. The same review surfacing the same issue after 40 hours of staging build work is a bug.

7-3. Engaging C2DAO as a Partner

The most effective data PMs treat C2DAO as a project partner, not a reviewer. This means:

- Briefing C2DAO on upcoming projects at the start of planning, not at the start of governance review.
- Sending C2DAO draft Ontology designs for informal feedback before formal submission, reducing the likelihood of formal review failure.
- Incorporating C2DAO's institutional knowledge of the MSS architecture into project planning — they often know about existing Object Types or datasets that can reduce build scope.
- Communicating project timeline changes that affect C2DAO review scheduling as soon as they are known.

C2DAO's formal role is reviewer. Their informal role, when the PM engages them correctly, is collaborator. The distinction is consequential for delivery velocity.

SECTION 8: PORTFOLIO VISIBILITY ON MSS

8-1. What the PM Can See in Foundry

Foundry provides native tooling for portfolio visibility that the PM should use before building any external tracking system. Key visibility surfaces:

Pipeline Lineage. Foundry's lineage graph traces the dependency chain from source to application for any dataset or Object Type. The PM can navigate this graph to understand what feeds what and identify single points of failure in the dependency chain. Use lineage as the primary tool for dependency mapping.

Dataset Health Indicators. Foundry surfaces pipeline execution status, last refresh time, and build errors at the dataset level. The PM monitoring availability health should build this view into a regular check. Pay attention to datasets that show "last updated" times significantly older than their scheduled refresh cadence — this is the earliest signal of a slow break.

Branch and Promotion History. Foundry records branch creation, merge, and promotion events. The PM can review this history to understand what was built when, who promoted it, and whether governance gates were followed. This is also the primary audit trail for compliance reviews.

Workshop Application Usage Metrics. Foundry Workshop surfaces application access frequency and active user counts. The PM monitoring utility health should pull these metrics monthly and compare against expected usage baselines. A 50% drop in application access with no corresponding change in the operational requirement is a signal worth investigating.

Checks and Build Logs. Data quality check results and transform build logs are accessible in Foundry and provide the primary evidence for quality health assessment. The PM does not need to interpret code-level logs but should be able to confirm that checks are running, passing rates are within thresholds, and failures are being routed to the responsible owner.

8-2. What the PM Cannot See in Foundry

Foundry tells the PM whether a product is running and how often it is accessed. It does not tell the PM whether the product is operationally valuable.

The PM must track these externally:

DRAFT

Gap	External Tracking Method
User satisfaction	Periodic user engagement sessions (quarterly minimum); direct feedback collection at delivery
Operational impact	Coordination with unit S6/G6/G2 to understand whether products are influencing decisions
Requirement currency	Scheduled requirements validation with product owners (every 90 days for tier-1 products)
Stakeholder priorities	Regular engagement with G3/G4/G6 staff to understand shifting operational focus
Personnel continuity	Maintained contact list for each product's builder, data steward, and technical owner

The PM who relies exclusively on Foundry-visible metrics for portfolio health will have a technically accurate but operationally incomplete picture. Complement platform visibility with direct stakeholder engagement.

8-3. Building a Portfolio View Without Additional Tooling

The PM does not need a custom dashboard to manage portfolio visibility. A well-structured spreadsheet maintained weekly is sufficient for most USAREUR-AF data programs. Columns:

Field	Purpose
Product Name	Official MSS name per naming convention
Tier	1 (mission-critical), 2 (operationally significant), 3 (administrative)
Owner	Named individual, reachable
Status	Running / Degraded / Failed
Health (Availability)	Green / Amber / Red
Health (Quality)	Green / Amber / Red
Health (Utility)	Green / Amber / Red
Debt Level	Low / Medium / High
Last Governance Review	Date
Open Issues	Count
Notes	Free text

Update this view weekly. Brief it to leadership monthly. This is the PM's primary portfolio management artifact. It should be able to answer any question a commander or senior leader asks about the state of the MSS data portfolio within 60 seconds.

SECTION 9: COMMON DATA PM FAILURE MODES

9-1. Overview

The following failure modes are observed consistently across data programs in theater environments. Each represents a predictable mistake — one that can be anticipated and avoided with the mental models developed in this guide.

9-2. Failure Mode 1: Requirements Without User Validation

Pattern. The PM receives data product requirements from a commander, G-staff section, or unit POC. The requirements are accepted as written. The product is built against those requirements. At delivery, actual users — the analysts, NCOs, and staff officers who will use the tool daily — have different needs than those captured by the requester.

Why it happens. Requesters and users are often different people. A G4 who requests a readiness dashboard may not be the staff officer who will use it at 0500Z every morning. Requirements captured from the top of an organization do not always reflect how the bottom of that organization works.

Prevention. Before accepting any requirements, conduct a user validation session with actual end users. Ask: what decisions will you make with this product? What does your current process look like without it? What would make you trust this product? These questions will surface requirements the formal request process missed.

9-3. Failure Mode 2: Committing to Timelines Before Data Access is Confirmed

Pattern. The PM commits to a delivery date before confirming that source system access is in place. Data access requests take longer than expected. The delivery date slips. Leadership loses confidence.

Why it happens. Schedule pressure arrives before data access is confirmed. The PM believes data access will resolve quickly. It does not.

Prevention. Treat data access confirmation as a project prerequisite, not a parallel track. Do not brief a delivery milestone to leadership until data access is either confirmed or a realistic access timeline has been established. The phrase "delivery date contingent on data access" is uncomfortable. A missed delivery date is worse.

9-4. Failure Mode 3: Managing to Schedule at the Expense of Quality

Pattern. The project is behind schedule. Quality checks and user acceptance testing are compressed or skipped to recover timeline. The product ships with defects. Operational users encounter data quality issues. Trust in the product — and in MSS broadly — is damaged.

Why it happens. Schedule pressure is visible and immediate. Data quality failures are delayed and diffuse. Under pressure, PMs optimize for the visible problem.

Prevention. Establish non-negotiable quality gates in the project plan before schedule pressure begins. When schedule slippage occurs, negotiate scope reduction rather than quality reduction. A smaller product delivered correctly is better than a full-scope product delivered wrong.

9-5. Failure Mode 4: Losing Track of Production vs. Development

Pattern. The portfolio grows. Multiple products are in various stages of development and production simultaneously. The PM loses precise clarity on which version of a product is in production, what changes are in flight, and what has been promoted versus what is still in a development branch. A development build is accidentally referenced in an operational brief.

Why it happens. MSS Foundry manages branches and promotions, but awareness of the portfolio state requires deliberate PM tracking. In a busy program, this tracking lapses.

Prevention. Maintain a production registry — a simple list of every data product currently in production, its version, its promotion date, and its current health status. Review and update this registry at each sprint close. Brief it to the team at sprint kickoff. No one should be uncertain about what is live.

9-6. Failure Mode 5: Failing to Retire Products

Pattern. A data product built for a specific operation — DEFENDER, a theater training event, a short-term operational requirement — is not retired when the operational requirement ends. It remains in production, consuming compute resources, accumulating governance debt, and appearing in pipeline lineage diagrams. Six months later, no one knows why it exists or whether it can be safely turned off.

Why it happens. Retirement requires a decision. Decisions require ownership. If no one is explicitly responsible for the retirement decision, the product persists by default.

Prevention. Every data product should have a documented lifecycle status: Active, Under Review, or Retiring. When an operational requirement ends, the PM initiates a retirement review: confirm no downstream dependencies, notify affected users, document the retirement in the governance record, remove the product from active monitoring. Retirement is not failure. An unmanaged portfolio is.

DRAFT

9-7. The Common Thread

Each of these failure modes shares a common root: the PM allowed urgency to override structure. Schedule pressure led to requirements shortcuts. Access uncertainty was deferred. Quality gates were compressed. Production tracking lapsed. Retirement decisions were avoided.

Structure is not bureaucracy. In a data program serving theater operations, structure is what ensures the readiness dashboard is right when the commander needs it, the logistics pipeline is clean when 21st TSC is counting on it, and the data product portfolio is accountable when the inspector general asks who owns what.

The PM's job is to hold that structure under pressure. That is what this guide has been preparing you to do.

SUMMARY: MENTAL MODEL REFERENCE

Concept	Core Principle
PM Role	Own outcomes, not artifacts. Accountable for a working, governed, operational portfolio.
Dependency Thinking	Every product is the end of a chain. Trace it. Map it. Know who owns each link.
Health vs. Status	Status = running. Health = availability + quality + utility. Check all three.
Milestone Risk	Data access, Ontology changes, governance delays, and user acceptance are predictable risks. Build them in.
Technical Debt	Identify it. Quantify it. Make it visible to leadership in operational risk terms.
Stakeholder Management	Translate between operational users and technical producers. Manage the delivery tension with staged delivery, transparent constraints, and scope negotiation.
Governance	Use checkpoints as milestone gates. Engage C2DAO as a partner before formal review begins.
Portfolio Visibility	Foundry shows availability and quality. User engagement shows utility. Track both.
Failure Modes	Requirements without user validation. Timelines before access. Quality sacrificed for schedule. Lost production clarity. Unretired products.

DRAFT

TRANSITION TO TM-40J TASK INSTRUCTION

This guide has developed the conceptual foundation for data program management on MSS. You should now be able to:

- Describe the PM's scope of accountability and how it relates to C2DAO governance.
- Trace a data product's dependency chain from source system to operational user.
- Distinguish between status and health and apply a three-dimension health framework.
- Build milestone risk into a project schedule before schedule pressure begins.
- Identify and quantify technical debt in a data portfolio.
- Translate between operational users and technical producers.
- Use governance checkpoints as milestone gates rather than schedule obstacles.
- Build portfolio visibility using native Foundry tooling supplemented by direct stakeholder engagement.
- Recognize and prevent the five most common data PM failure modes.

Proceed to TM-40J for task-based instruction on executing these responsibilities within the MSS environment.

CONCEPTS GUIDE — TM-40J COMPANION | DATA PROGRAM MANAGER | MAVEN SMART SYSTEM HQ USAREUR-AF, Wiesbaden, Germany | 2026 DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

DRAFT